RICE UNIVERSITY

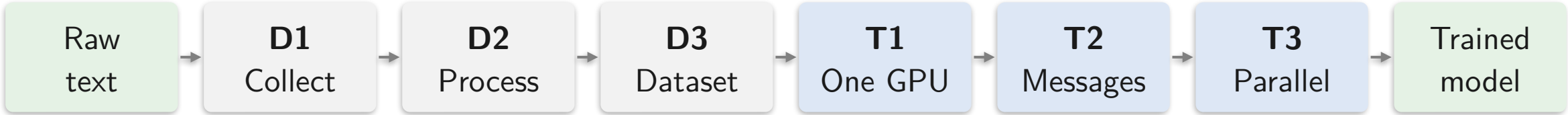
The AI Pipeline, End to End

Every system between raw text and a trained model, and why each one exists

Christopher Jermaine and Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 1, Lecture 1



A puzzle to start

A friend says: "Download Wikipedia, call `model.fit(wikipedia)`, done." What goes wrong?

- **It is not text yet**: one $\approx$ 25 GB compressed XML file, full of markup. $\rightarrow$ **D1**
- **It is too big to clean quickly** on one machine. $\rightarrow$ **D2**
- **It has copies**, test articles can leak in, and models read **tokens**. $\rightarrow$ **D3**
- **The arithmetic is too much** for a CPU: you need a GPU and a framework. $\rightarrow$ **T1**
- **One GPU is too slow and too small**: many GPUs must talk and share. $\rightarrow$ **T2, T3**

Key idea

Five problems, five systems, six assignments. Today: **why** each one exists.

Source: Wikipedia:Database download, accessed Nov 2026 (size of the English articles dump).

Learning goals for today

- **Trace** one workload from raw text to a trained model, naming each stage and what it hands to the next.
- **Explain** the problem each stage solves: what goes wrong without it.
- **Estimate** the data, compute, time, and cost of a real training job, to within a factor of a few.
- **Describe** how the course works: two blocks, six assignments, a capstone, and predict $\rightarrow$ build $\rightarrow$ measure.

How much text does a real model read?

Llama 2 was trained on $\mathbf{2 \times 10^{12}}$ **tokens**. A **token** is a piece of a word: the unit a model reads.

How many copies of English Wikipedia is that?

English Wikipedia
$\approx$ 7 million articles, $\approx$ 5 billion words
$\approx 7 \times 10^9$ tokens

$\mathbf{2 \times 10^{12}}$ **tokens**
$\approx$ **300 Wikipedias**
$\approx$ **8 TB** of plain text

Key idea

A real model reads **hundreds of Wikipedias**. Just collecting and cleaning that text takes systems: the data block.

Source: Touvron et al. 2023 (Llama 2); Wikipedia:Size of Wikipedia (Nov 2026); 1 token $\approx$ ¾ word $\approx$ 4 characters (assumed); copies and TB derived.

How much compute, time, and money?

FLOP: one floating-point add or multiply. **Parameter**: one number the model learns.

$$C = 6 \cdot \Psi \cdot D = 6 \times 7.5 \times 10^{9} \times 2 \times 10^{12} \approx 9 \times 10^{22} \text{ FLOPs}$$

$$\text{one GPU at 40\% of peak:} \quad \approx 7.2 \text{ years}$$

$$\approx 63{,}000 \text{ GPU-hours} \quad \approx \$130{,}000 \quad \text{(at an assumed \$2 per GPU-hour)}$$

Key idea

With 1,024 GPUs: $\approx$ **2.6 days**, but only if the systems let them work together efficiently: the training block.

Source: Kaplan et al. 2020 (6ND); ZeRO, SC 2020 (Ψ); Llama 2 (D); H100 datasheet (BF16 dense); 40% and \$2/GPU-hour assumed; times and cost derived.

What “systems for AI” means

Model code
a few dozen lines
our MLP: 34,338 parameters

runs on →

Files and formats: XML, Parquet, token shards

Parallel data processing: Spark

Frameworks and compilers: PyTorch, Triton

GPUs and their memory

Communication: MPI, NCCL — many GPUs

Key idea

A **system** stores, moves, and computes on data for you, so model code stays short. Its design sets speed, cost, and what is possible.

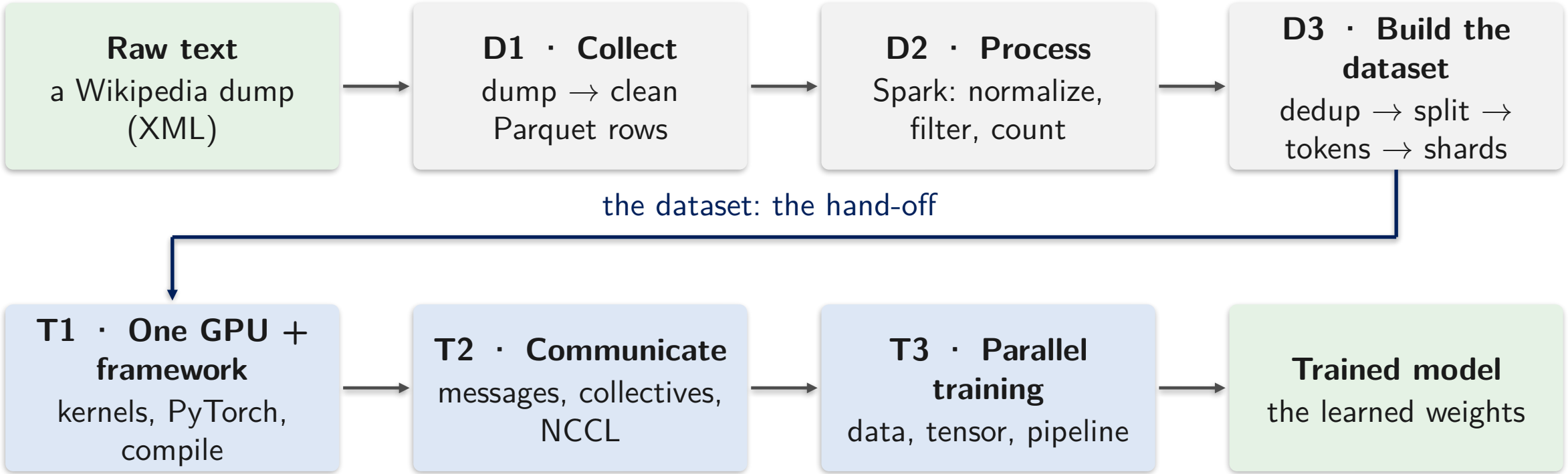
Part 1

The pipeline in one picture

What happens between raw text and a trained model?

From raw text to a trained model

Data systems · Weeks 2–7 · Jermaine



Training systems · Weeks 8–13 · Wang

Each box exists because of one problem. Today: that problem.

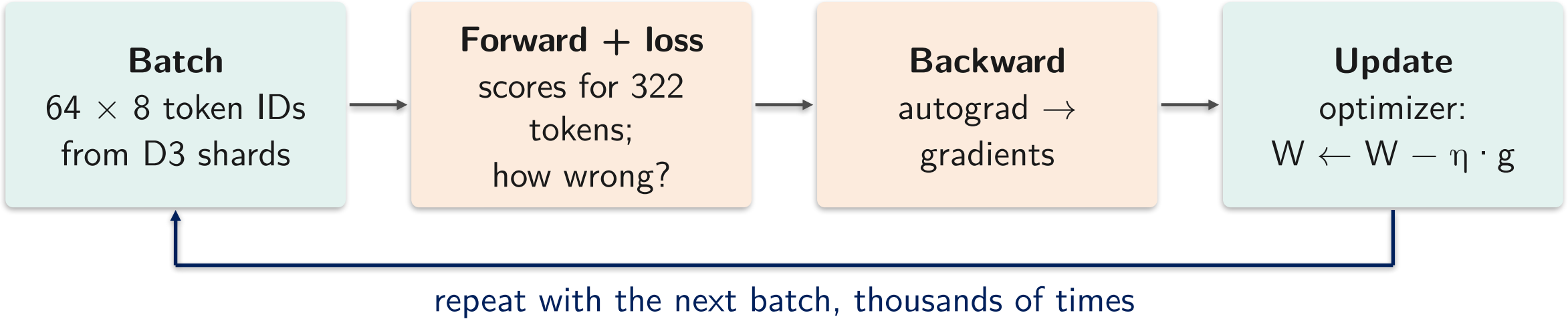
Follow one article through the pipeline

Raw dump	→	`<page><title>Arrays</title><ns>0</ns><id>11</id>...`
D1	→	`doc_id="11", title="Arrays", text="Arrays store..."`
D2	→	normalized text, kept or dropped with a reason (“short”, …)
D3	→	`[256, 65, 114, 114, 97, 121, 115, 32, ..., 257]`
T1–T3	→	8 tokens of context → predict the next one; batches of 64

Each stage changes the **form** of the data: XML → table rows → clean text → integers → batches.

Source: D1–D3 handouts, Spring 2027 (example page, quality rules, BOS = 256 / EOS = 257); course MLP config.

What one training step does



In our pipeline

Training time = steps × time per step. Weeks 8–10 make **one step fast**; Weeks 10–13 split it across **many GPUs**.

Check yourself

Two minutes with a neighbour.

- Put in order: token shards, an XML dump, Parquet rows, trained weights, filtered text.
- Which stage's output is the hand-off from the data block to the training block?
- In one training step, which phase computes the gradients, and what tool provides it?

Part 2

The data systems

Why can't we feed raw text straight to the GPU?

Why do we need a collection step?

The input: one compressed XML file
≈ 25 GB compressed, ≈ 105 GB unpacked
articles, redirects, talk pages, markup

Why not just read it all into a Python list?

105 GB does not fit in a laptop's **16 GB** of memory, and most of it is not article text.

D1 Collect does

Stream: one page at a time

Strip markup; skip and **log** non-articles

Write typed rows to **Parquet**

Keep **license** and **source**

In our pipeline

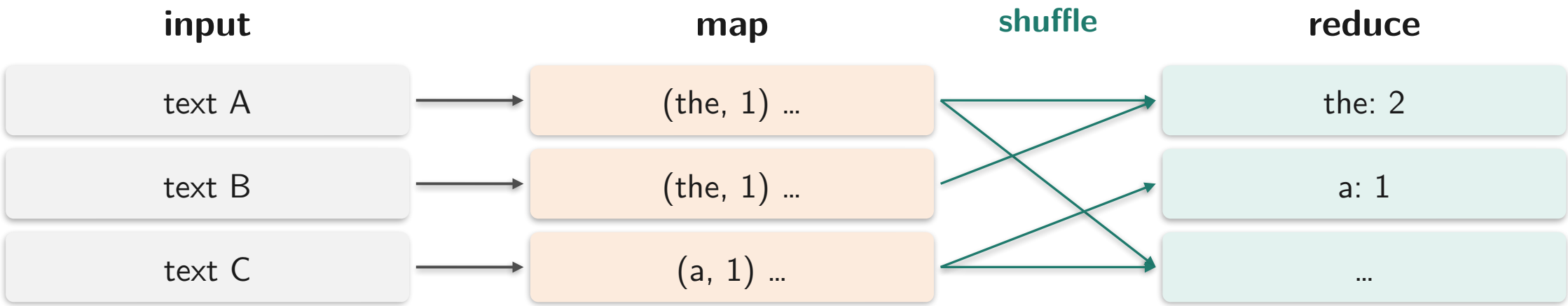
Why collect? A model learns only from text you may use and can read quickly. D1 (due Week 3) turns the dump into clean rows.

Source: Wikipedia:Database download, accessed Nov 2026 (approximate); laptop memory assumed; D1 handout.

Why process data on many machines?

≈ 8 TB of text, at ≈ 10 MB/s per core (assumed): how long on one core?

≈ 9 days. (Wikipedia alone: ≈ 3 hours.)



Key idea

Split into **partitions**, one per worker: 1,000 cores ≈ **13 minutes**. The price: the **shuffle** moves data, and machines fail.

Source: Dean & Ghemawat, OSDI 2004 (MapReduce); 10 MB/s per core assumed; times derived.

Process (D2): Spark splits the work for you

```
df = spark.read.parquet("articles.parquet")
kept = df.filter(F.length("text") >= 120)
w = F.explode(F.split(F.lower("text"), r"\W+"))
top = (kept.select(w.alias("word"))
           .groupBy("word").count()    # shuffle
           .orderBy(F.desc("count")))
top.show(12)                           # now it runs
```

- You say **what**; Spark decides **how**: partitions, stages, tasks.
- **Lazy**: nothing runs until an action (`show`, `count`, a write).
- **D2**: apply the given rules, count, and match a one-machine run.

In our pipeline

Why process with Spark? A raw corpus is too large and messy to clean on one machine. D2 (due Wed, Week 5) spreads the cleaning over many.

Source: Zaharia et al., NSDI 2012; Armbrust et al., SIGMOD 2015; D2 handout (120-character rule).

Why remove duplicates?

The problem

One **61-word sentence** appears **over 60,000 times** in C4, a web corpus.

Models trained on copies **memorize** them, and waste compute.

Why not compare every pair of articles?

$$n(n-1) \,/\, 2 \approx 2.6 \times 10^{13} \text{ pairs} \quad \approx 300 \text{ days at } 10^6 \text{ pairs/s}$$

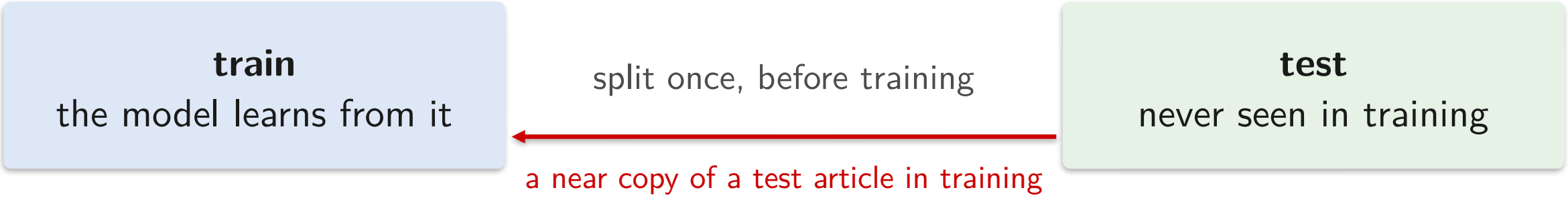
MinHash + LSH: compare only documents whose short signatures share a bucket.

In our pipeline

Why deduplicate? Copies distort training; LSH finds them without n^2 comparisons. D3's checkpoint (Week 6).

Source: Lee et al., ACL 2022; Wikipedia:Size of Wikipedia (n); 10^6 pairs/s assumed; pairs and days derived.

Why must test data never leak?



Then the test score measures **memory**, not learning. Overlap like this affects **over 4%** of standard validation sets.

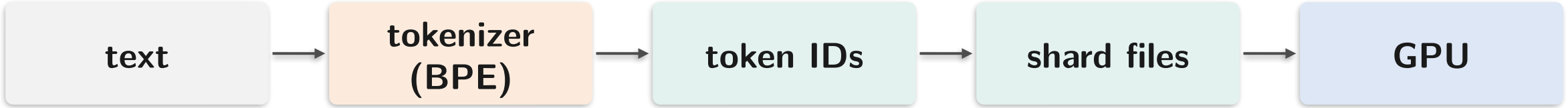
Fix: **deduplicate first**, then split by a hash of each document ID; check that nothing crosses.

In our pipeline

Why a leak-free split? Otherwise you cannot trust the test score. D3 checks for zero leaks.

Source: Lee et al., ACL 2022 (train–test overlap); D3 handout (split rule and leak check).

Why tokens and shards?



Token: an integer ID for a piece of text. **Shard**: one file of a dataset split into many.

D3's shards are flat arrays of **2-byte** integers, so a loader can read them as an array.

$$2 \times 10^{12} \text{ tokens} \times 2 \text{ bytes} = 4 \text{ TB of shards}$$

$$1{,}024 \text{ GPUs read} \approx 9 \text{ million tokens per second}$$

In our pipeline

Why tokens and shards? The GPU must never wait for text processing: tokenize **once**, offline. D3 (due Week 7).

Source: Sennrich et al., ACL 2016 (BPE); D3 handout (uint16 shards); H100 datasheet; 40% assumed; rates derived.

Check yourself

Two minutes with a neighbour.

- Why does D1 stream the dump instead of loading it into memory first?
- A test article has a near copy in the training set. What goes wrong, and which step prevents it?
- Why is comparing every pair impractical for 7 million articles?

Part 3

The training systems

Why can't one GPU and a for-loop train the model?

Training systems at a glance

Problem	System component	Where
One step is millions of multiply-adds	**GPUs and kernels**	Wk 8 · T1
A kernel per op is impractical	**Frameworks and compilers**	Wk 9–10 · T1
One GPU is too slow or too small	**Processes + communication: MPI, NCCL**	Wk 10–11 · T2
How do we split the work?	**Data, tensor, pipeline parallelism**	Wk 12–13 · T3
Things break at scale	**Diagnosis**	Wk 13

Why GPUs and kernels?

The same job, 9×10^{22} FLOPs, almost all multiply-adds:

One **CPU core**, at its peak
$\approx$ **15,000 years**

One **H100 GPU**, at 40% of peak
$\approx$ **7.2 years**

Kernel: one function the GPU runs on many elements at once; the CPU only **launches** it.

In our pipeline

Why GPUs? Thousands of multiply-adds at once. T1: write one GPU kernel in Triton (Week 8; due Week 10).

Source: H100 datasheet (SXM5), BF16 dense; CPU core assumed (3 GHz, 2 AVX-512 FMA units); 40% assumed; times derived.

Why frameworks and compilers?

```
model = MLP().to("cuda")
opt = torch.optim.AdamW(model.parameters())
for x, y in loader:           # D3 shards
    x, y = x.cuda(), y.cuda()
    loss = F.cross_entropy(model(x), y)
    loss.backward()           # autograd
    opt.step(); opt.zero_grad()
fast = torch.compile(model)   # fewer kernels
```

- **Framework** (PyTorch): tensors on the GPU, GPU ops, **autograd**.
- Every line launches **GPU kernels** for you.
- **Compiler** (`torch.compile`): fuses many small ops into **fewer kernels**.

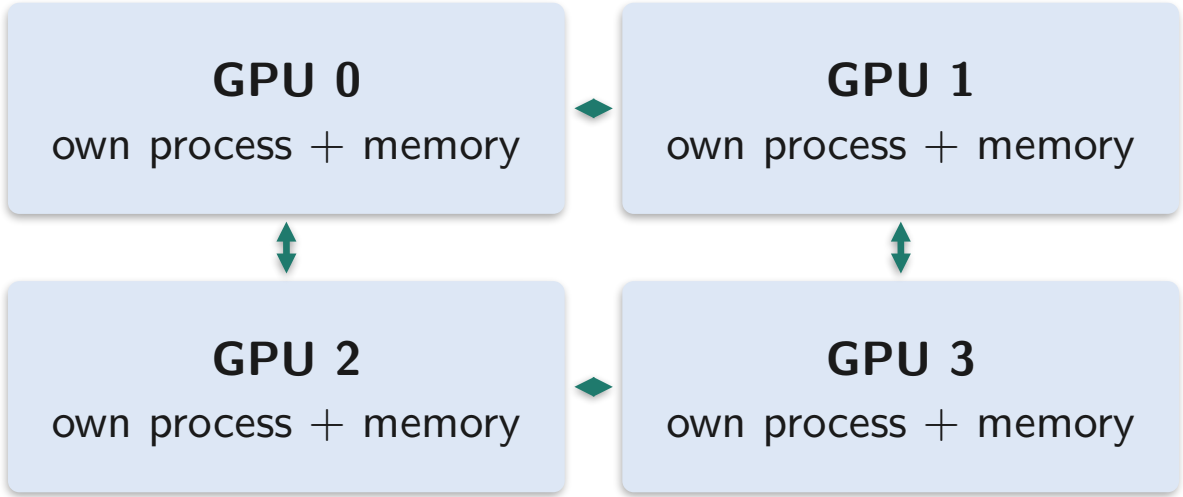
In our pipeline

Why frameworks? Nobody writes a kernel and a gradient for every operation. Weeks 9–10; T1 compares eager and compiled.

Source: PyTorch documentation: autograd, torch.compile.

Why communication?

Training state of the 7.5×10^9-parameter model: **120 GB**. One H100: **80 GB**.



Every step: sum **15 GB** of gradients across all GPUs.

Collective: a step every process takes part in, e.g. **all-reduce** (sum, then everyone gets it).

MPI (CPUs) and **NCCL** (GPUs) do this.

In our pipeline

Why communication? One GPU is too small and too slow; many GPUs must **exchange data** every step. T2 (Weeks 10–11).

Source: ZeRO, SC 2020 (16 bytes per parameter); H100 datasheet (80 GB); 120 GB and 15 GB derived.

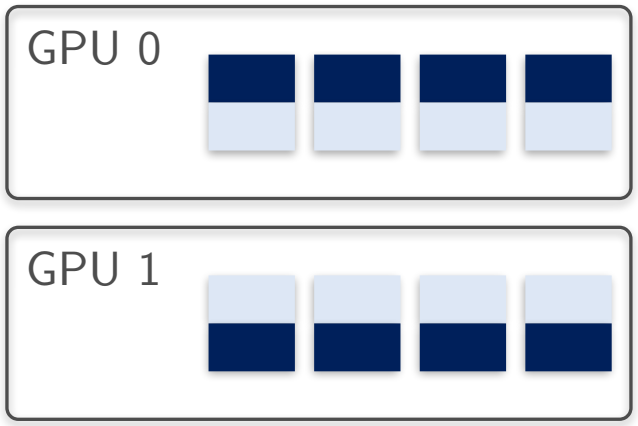
Why parallel training?

Data parallelism



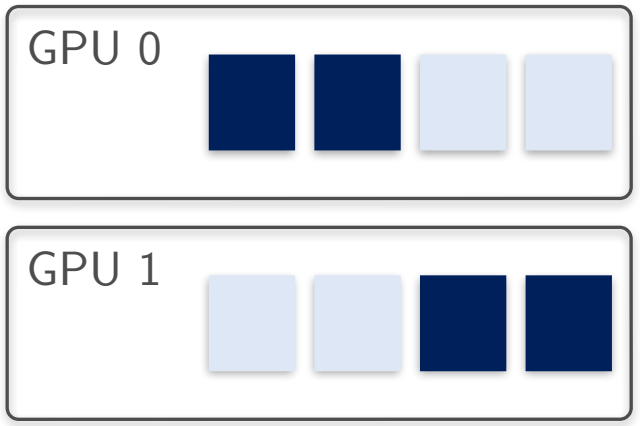
each GPU: whole model, **different examples**

Tensor parallelism



each GPU: **part of every matrix**

Pipeline parallelism



each GPU: **some of the layers**

In our pipeline

Why parallelism? 2.6 days on 1,024 GPUs only if the split keeps memory and communication in check. T3 (Weeks 12–13).

Source: Narayanan et al., SC 2021 (Megatron-LM); ZeRO, SC 2020; 2.6 days derived (opening slide).

Why diagnosis?

Llama 3: up to 16,384 GPUs for 54 days
419 unexpected interruptions: one every $\approx$ **3 hours**

- One **slow GPU** makes every other GPU wait at every step.
- One **mismatched call** can hang the whole job, silently.
- Tools: **traces**, timeouts, checkpoints, and reproducible measurements.

In our pipeline

Why diagnosis? At scale, something is always broken or slow, and the cause is hard to see. Week 13.

Source: Dubey et al., "The Llama 3 Herd of Models," 2024, §3.3.4; interval derived.

Check yourself

Two minutes with a neighbour.

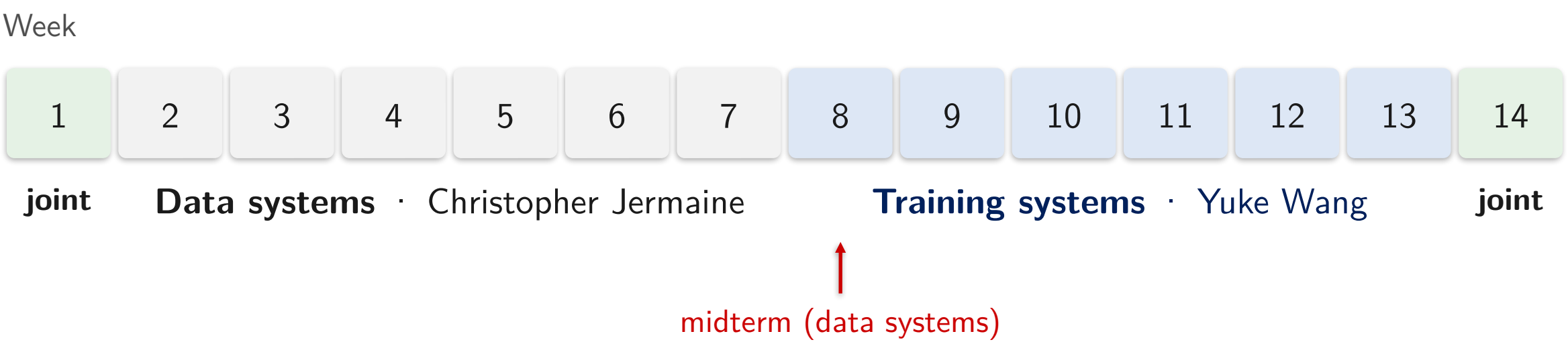
- What does a framework give you that a GPU alone does not?
- Training state: 120 GB. GPU memory: 80 GB. Name two ways out.
- Why does a 1,024-GPU job need diagnosis tools that a one-GPU job does not?

Part 4

How the course works

What will you build, and how will you know it works?

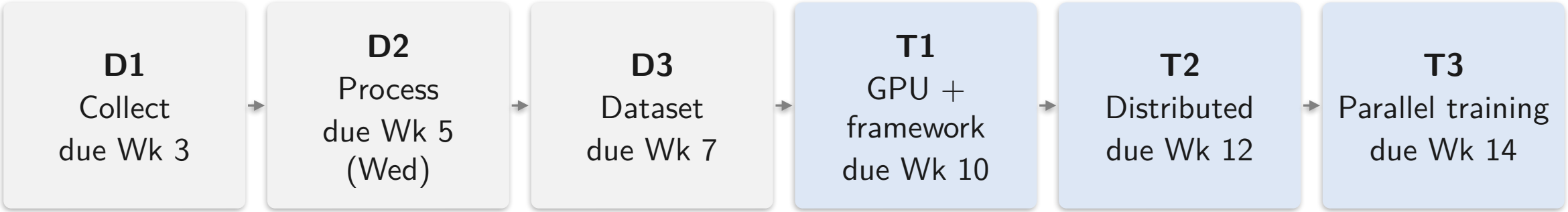
Two blocks, two instructors



- Each week: **two lectures** (with notebooks), **one recitation**, **one concept check**.
- Weeks 1 and 14 bring both blocks together; the final exam is cumulative.

Source: COMP 329 syllabus, Spring 2027: Course Schedule; How the Course Works.

Six assignments and a capstone

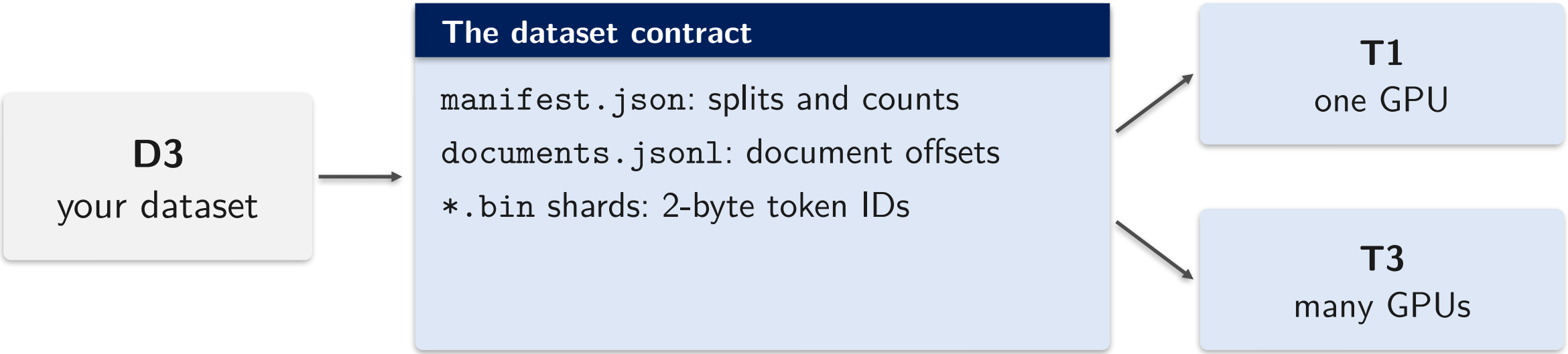


Capstone, in pairs (due Tue, May 4)
one systems question on the shared pipeline: predict, run two controlled experiments, explain

Each assignment: you write **one selected component**; staff provide the model, autograd, optimizers, and reference programs. **Labs 1–5** are short and guided.

Source: COMP 329 syllabus, Spring 2027 (Assessment; Course Schedule); COURSE_SEQUENCE.md.

The hand-off: the D3 dataset



(T2 uses synthetic tensors to isolate communication.)

You are never blocked

Fell behind? **Reference outputs** come out 48 hours after each deadline: start the next stage from them.

Source: COMP 329 syllabus, Spring 2027 (dataset contract; late work); D3 handout.

Predict, build, measure

1. Predict
estimate bytes, FLOPs, memory, time **first**

→

2. Build
implement one component; pass the public tests

→

3. Measure
at scale; explain the gap from your prediction

D1 example: will 2 workers help on a 37-page file? Build the streaming collector. Measure 1, 2, 4 workers; time decompression alone.

Key idea

A gap between prediction and measurement is not a failure: it **tells you what you missed**.

Source: COMP 329 syllabus, Spring 2027 (predict, build, measure); D1 handout (study).

Check yourself

Two minutes with a neighbour.

- Which assignment's output do T1 and T3 read, and what fixes its format?
- What are the three steps of every assignment?
- Your D2 is not finished at the deadline. How do you still start D3?

IN-CLASS BUILD

Where does the time go?

Pairs, on paper, with numbers from today's slides.

Exercise: estimate one pipeline run

Stage	Given	Your estimate
1. D1 parse	105 GB of XML at 10 MB/s per core	1 core? 16 cores?
2. D3 shards	2×10^{12} tokens, 2 bytes each	total bytes?
3. T3 train	9×10^{22} FLOPs; 1,024 GPUs at 4×10^{14} FLOP/s	days?

THEN

Assume perfect scaling. Which stage dominates the time, and which assumption is the most optimistic?

STRETCH

Dedup by comparing every pair of 7 million articles at 10^6 pairs per second: how long?

Exercise: solutions

Stage	Estimate	Note
1. D1 parse	$\approx$ 3 hours; $\approx$ 11 min on 16 cores	if decompression splits
2. D3 shards	4×10^{12} bytes $= 4$ TB	half the text's size
3. T3 train	$\approx 2.2 \times 10^{5}$ s ≈ 2.5 days	if communication is free

Stretch: $\approx 2.6 \times 10^{13}$ pairs $\approx$ **300 days**: longer than training.

Key idea

Training dominates the time, but a poor **algorithm** can make any stage dominate. Lecture 2: estimating, properly.

Source: Numbers from this lecture's slides; 10 MB/s per core, 10^6 pairs/s, 40% of peak assumed; all estimates derived.

Key takeaways

- A real model reads **hundreds of Wikipedias** and needs **years** of one GPU's time.
- **Data systems** make the text usable: collect (D1), process in parallel (D2), dedup, split, tokenize, shard (D3).
- **Training systems** make the step fast and shared: GPUs and frameworks (T1), communication (T2), parallelism (T3).
- The **D3 dataset** is the hand-off from the data block to the training block.
- Every assignment: **predict, build, measure**, and explain the gap.

Next time, and readings

When	What
Lecture 2 (this week)	Measurement: bytes, FLOPs, time, dollars, joules
This week	Lab 1 out: environment, a cluster job, CPU matrix multiplication
Friday	D1 released; dataset contract published
Week 2	Collecting and storing raw data (Jermaine)

READ BEFORE WEEK 2

Dean & Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," OSDI 2004, §1–3.

Syllabus: "The System You Will Study and Build".